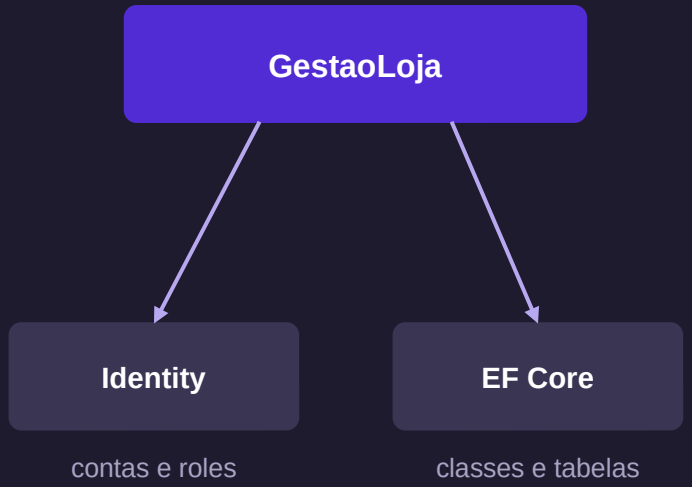
PROGRAMAÇÃO WEB · 2026/27 · AULA LABORATORIAL

Ficha 3
App Gestão de Loja

A primeira aplicação do trabalho prático: um projeto com contas de utilizador, o modelo de dados da loja e o código de arranque.



PLANO DA AULA

Como lá vamos chegar

1

Criar o projeto

Alínea 1.a. Aplicativo Web Blazor com Contas Individuais.

2

Explorar o template

Alínea 1.b. Base de dados, EF Core, migrations e a pasta Data.

3

Relações no EF Core

Etapa 2. O vocabulário e as quatro convenções 1-N.

4

Entidades e contexto

Alíneas 3.a a 3.d. As três entidades, os DbSet e o utilizador.

5

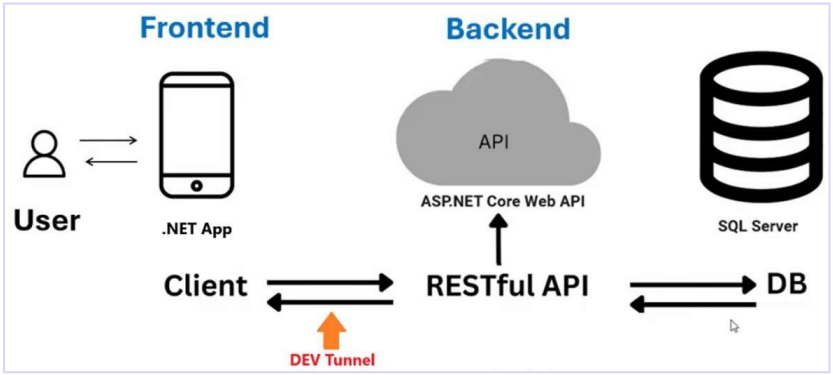
Configurar o arranque

Alíneas 3.e a 3.n. Connection string, roles e administrador.

Como usar os materiais: estes slides explicam o contexto e os conceitos. O guia HTML da ficha 3 tem o passo a passo, alínea a alínea, com o código completo para copiar e a indicação exata de onde fica cada ficheiro.

CONTEXTO

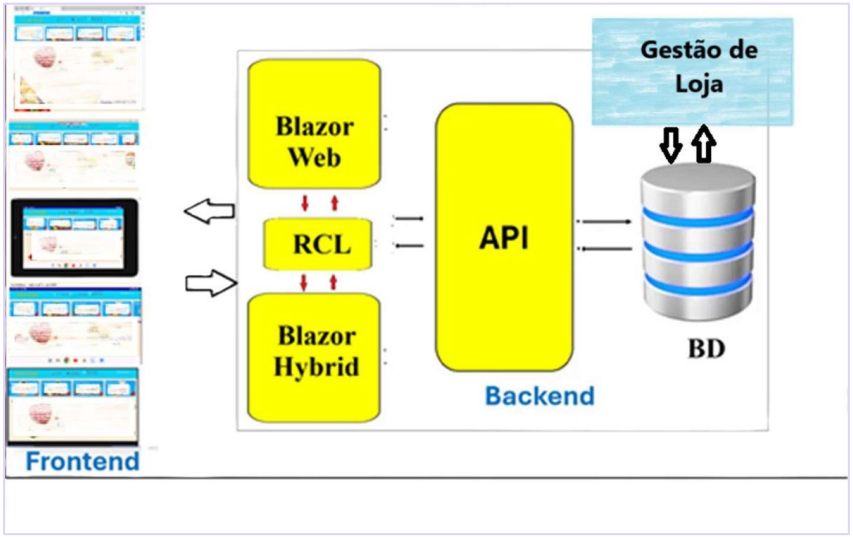
A arquitetura do trabalho prático



O caminho de um pedido do cliente: aplicação, API e base de dados.

Porque fica de fora da primeira figura

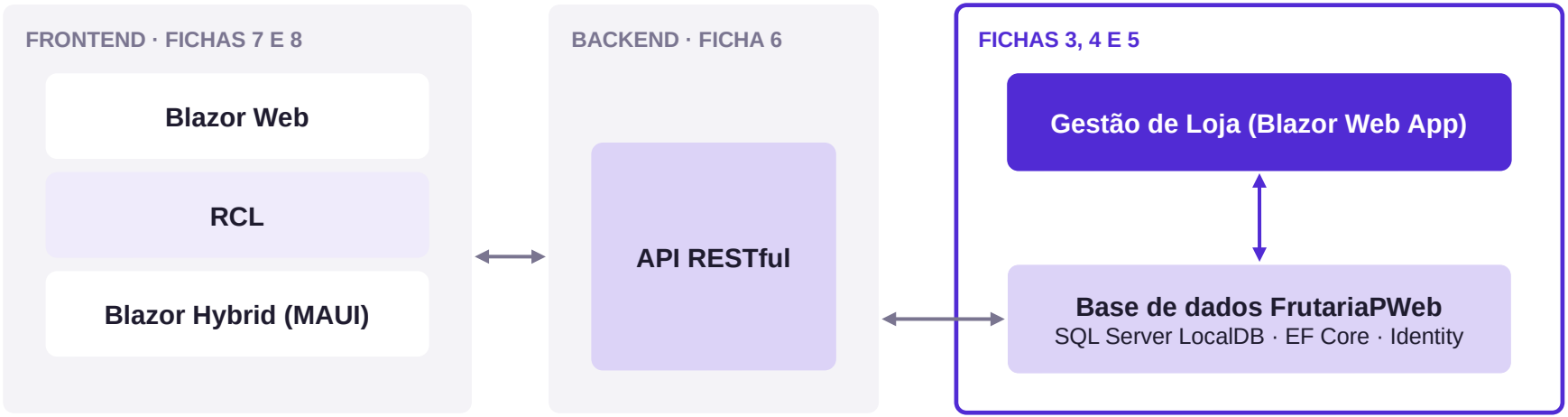
A Gestão de Loja não é usada pelo cliente da loja. É uma aplicação de gestão interna: quem a usa é o administrador.



A imagem completa: a Gestão de Loja liga-se diretamente à base de dados.

CONTEXTO

Onde encaixa a Gestão de Loja



Frontend e backend no mesmo projeto

As páginas e o acesso aos dados vivem juntos. Não há API pelo meio: o C# das páginas usa diretamente o contexto de base de dados.

É ela que cria a base de dados

A API da ficha 6 e o frontend das fichas 7 e 8 vão ler os produtos e as categorias que forem registados aqui.

CONTEXTO

A Gestão de Loja em três fichas

Ficha 3 · hoje

A infraestrutura

- Projeto com Contas Individuais
- Três entidades e os seus DbSet
- Connection string
- Roles e administrador iniciais
- AppConfig e imagens

No fim: compila, mas ainda não arranca.

Ficha 4

A base de dados

- Add-Migration BDInicial
- Update-Database
- Conferir as tabelas criadas no SQL Server

No fim: arranca e cria sozinha o administrador.

Ficha 5

As páginas

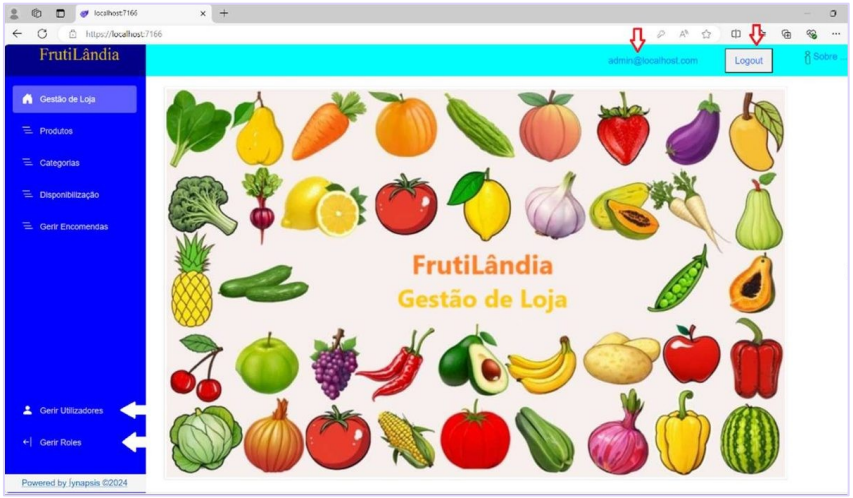
- Home, Sobre, layout e menu
- Páginas de gestão (CRUD) geradas a partir das entidades
- Login como administrador
- Inserir categorias, modos de entrega e produtos

No fim: aplicação completa.

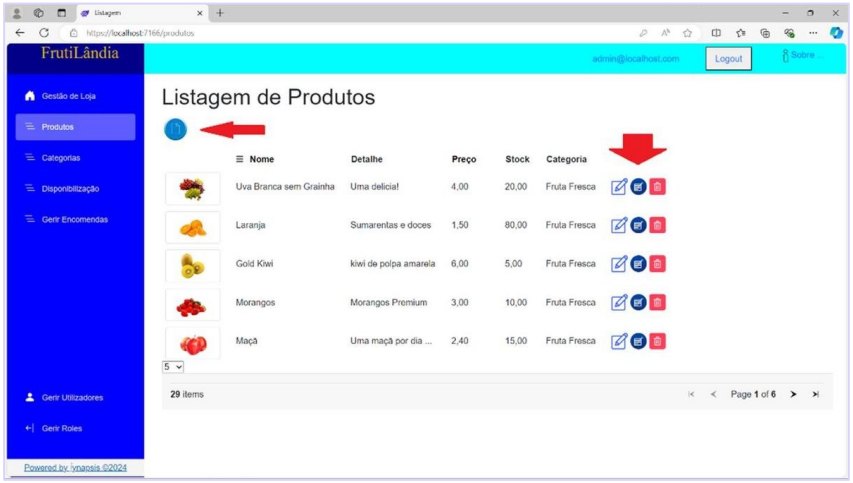
Porquê três fichas: primeiro as classes, depois as tabelas criadas a partir delas, por fim as páginas que as usam.

CONTEXTO

O destino: a aplicação no fim da ficha 5



Página inicial depois do login como administrador.



Listagem de produtos, com os botões de editar, ver e apagar.

Hoje não se vê nada disto. Preparamos o que fica por baixo: quem é o utilizador, que dados existem e como se relacionam.

PONTO DE PARTIDA

O que fica feito hoje, e o que ainda não

Fica feito na ficha 3

- Projeto GestaoLoja com registo e login
- Três classes de domínio na pasta Entities
- Contexto de base de dados com três DbSet
- Utilizador com nome, morada, NIF e estado
- Ligação à base de dados FrutariaPWeb
- Código que cria os roles e o administrador
- Imagens da interface em wwwroot/img

Ainda não existe

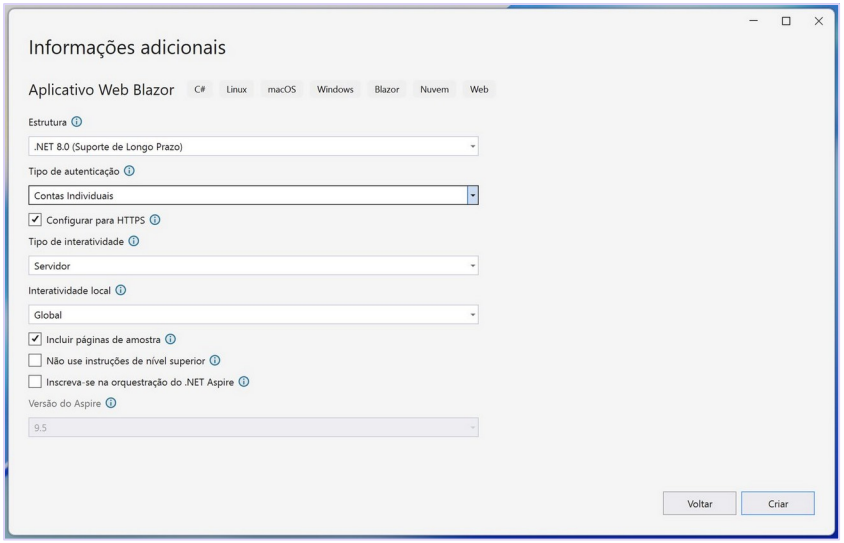
- A base de dados e as tabelas (ficha 4)
- As páginas de gestão de categorias, produtos e modos de entrega (ficha 5)
- O menu e as páginas Home e Sobre finais (ficha 5)
- Encomendas, gestão de utilizadores e de roles (mais tarde)

Atenção: a meio da ficha a aplicação deixa de arrancar

A partir da alínea 3.k o arranque consulta a base de dados, que só nasce na ficha 4. Não é um erro. Até lá, a ficha pede apenas para compilar.

ALÍNEA 1.a · CRIAR O PROJETO

Criar o projeto: Aplicativo Web Blazor



Ecrã Informações adicionais (Figura 3 da ficha).

1. **Nome: GestaoLoja**
   Passa a ser o namespace de todo o código.

2. **.NET 8.0**
   A versão usada em todas as fichas.

3. **Contas Individuais**
   A novidade: registo e login, guardados na nossa base de dados.

4. **Servidor e Global**
   Como na ficha 2: C# no servidor, todas as páginas interativas.

Vem com "Nenhum" e não dá para mudar depois: sem Contas Individuais, cria-se o projeto de novo.

ALÍNEA 1.b · O QUE O TEMPLATE TROUXE

Contas Individuais: o que muda no projeto

GestaoLoja/ (pasta do projeto)

```
├─ Components/
│   ├─ Account/              login, registo, ...
│   ├─ Layout/
│   └─ Pages/
│       ├─ Auth.razor        só com sessão
│       └─ Home, Counter, Weather
├─ Data/
│   ├─ Migrations/           tabelas de utilizadores
│   ├─ ApplicationDbContext.cs
│   └─ ApplicationUser.cs
├─ wwwroot/
├─ appsettings.json      + connection string
└─ Program.cs            + autenticação e EF Core
```

Com "Nenhum" (ficha 2)

Sem pasta Data, sem base de dados e sem login. Qualquer pessoa vê todas as páginas.

Com "Contas Individuais"

O template traz os pacotes do EF Core e do Identity, a pasta Data, as páginas de conta, e as entradas Register e Login no menu.

A laranja: o que não existia no projeto Blazor Web da ficha 2.

ALÍNEA 1.b · IDENTITY

Identity: quem és e o que podes fazer

Autenticação: quem és?

Registo e login com email e palavra-passe. Depois do login, um cookie identifica o utilizador em cada pedido.

Autorização: o que podes fazer?

Cada utilizador tem roles. Na loja: Admin, Gestor e Cliente. As páginas decidem a quem se mostram.

ASP.NET Core Identity: uma biblioteca da Microsoft, parte do ASP.NET Core. Não a escrevemos nem a instalamos à mão: ao escolher Contas Individuais, o template já a traz configurada.

O que nos entrega, já feito

Tabelas

As sete tabelas AspNet da base de dados. A palavra-passe não é guardada: guarda-se o seu hash.

Serviços

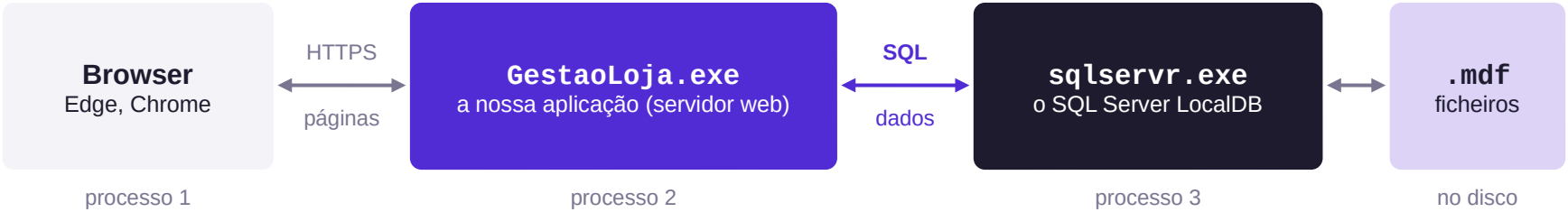
UserManager, SignInManager e RoleManager: classes prontas para criar utilizadores, iniciar sessão e gerir roles.

Páginas

Registo, login e gestão de conta, na pasta Components/Account do projeto.

ALÍNEA 1.b · BASE DE DADOS

A aplicação e a base de dados são dois programas



Um servidor de base de dados é um programa

O SQL Server é um processo que fica à escuta de ligações. Recebe comandos SQL, executa-os e devolve os resultados. Só ele mexe nos ficheiros de dados.

A aplicação é cliente dele

A GestaoLoja não abre os ficheiros da base de dados. Liga-se ao servidor e pede-lhe o que precisa. A connection string diz-lhe onde ele está e como entrar.

O que o LocalDB tem de especial

É um SQL Server reduzido, para desenvolvimento. Não fica sempre ligado: arranca sozinho quando chega a primeira ligação e corre com a conta do utilizador do Windows.

ALÍNEA 1.b · EF CORE

ORM: das classes às tabelas



FICHA 3 | FICHA 4

O que é um ORM

Object-Relational Mapper. Trabalha-se com objetos C#, e a biblioteca (o Entity Framework Core) traduz para SQL e devolve objetos.

A consequência

A base de dados é um espelho das classes. Um tipo ou uma relação mal escritos na classe passam para a tabela.

O que escrevemos (LINQ, como na ficha 1)

```
context.Produtos
    .Where(p => p.Preco < 5).ToList();
```

O que o EF Core envia à base de dados

```
SELECT * FROM Produtos
WHERE Preco < 5
```

ALÍNEA 1.b · ARQUITETURA

Da página aos dados: onde fica cada peça



A página

A vista. Mostra dados ao utilizador e recolhe o que ele escreve nos formulários.

As entidades

Os nossos modelos de dados: uma classe por cada coisa que a loja guarda.

O contexto

A porta para a base de dados: recebe e entrega entidades, e traduz para SQL.

As tabelas

Onde os dados ficam guardados, mesmo com a aplicação parada.

ALÍNEA 1.b · ENTIDADES

Entidades: uma classe, uma tabela

Entities/Categoria.cs (abreviado)

```
public class Categoria
{
    public int Id { get; set; }
    public string? Nome { get; set; }
    public int Ordem { get; set; }
}
```

migrations

(ficha 4)

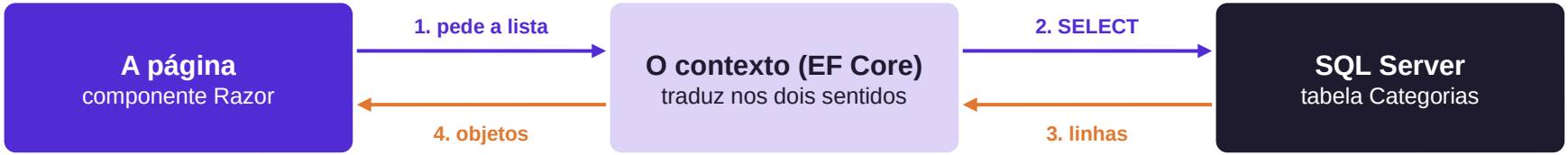
Tabela `Categorias`

Id	Nome	Ordem
1	Fruta Fresca	1
2	Frutos Secos	2
3	Sumos de Fruta	3

No C#	Na base de dados	No exemplo
a classe (entidade)	a tabela	`Categoria` e `Categorias`
uma propriedade	uma coluna	`Nome` e `Nome`
o tipo da propriedade	o tipo da coluna	`int` e `int`; `string` e `nvarchar`
um objeto	uma linha	`new Categoria { ... }` e a linha 2
uma lista de objetos	várias linhas	`List<Categoria>` e a tabela inteira

ALÍNEA 1.b · ENTIDADES

Ler: das tabelas para objetos C#



O que a página escreve (C#)

```
context.Categorias.OrderBy(c => c.Ordem).ToList()
```

O que o EF Core envia (SQL, simplificado)

```
SELECT Id, Nome, Ordem FROM Categorias ORDER BY Ordem
```

O que a base de dados devolve

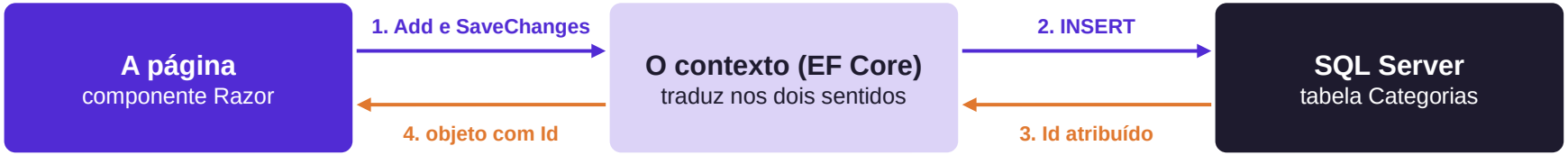
Id	Nome	Ordem
1	Fruta Fresca	1
2	Frutos Secos	2
3	Sumos de Fruta	3

O que a página recebe: List<Categoria>, um objeto por linha

```
Categoria { Id = 1, Nome = "Fruta Fresca",   Ordem = 1 }
Categoria { Id = 2, Nome = "Frutos Secos",   Ordem = 2 }
Categoria { Id = 3, Nome = "Sumos de Fruta", Ordem = 3 }
```

ALÍNEA 1.b · ENTIDADES

Gravar: de objetos C# para as tabelas



O que a página escreve (C#)

```
var c = new Categoria { Nome = "Compotas", Ordem = 4 };
context.Categorias.Add(c);
await context.SaveChangesAsync();
```

O que o EF Core envia (SQL, simplificado)

```
INSERT INTO Categorias (Nome, Ordem)
VALUES ('Compotas', 4)
```

Nada acontece antes do SaveChanges

Add só regista a intenção. É o SaveChanges que envia o SQL à base de dados.

O Id vem da base de dados

O objeto parte com Id 0. Depois de gravado, o EF Core preenche-o com o Id atribuído.

Alterar e apagar

Muda-se a propriedade, ou chama-se Remove. O SaveChanges envia o UPDATE ou o DELETE.

ALÍNEA 1.b · EF CORE

Code first: as classes mandam na base de dados

Code first: a nossa abordagem

Classes C# escrevemos → **Migrations** o EF gera → **Base de dados** o EF cria

Quando se usa

Em projetos novos. A estrutura da base de dados fica escrita no código, vai para o Git, e qualquer pessoa a recria na sua máquina.

Database first

Base de dados já existe → **Scaffold** o EF lê → **Classes C#** o EF gera

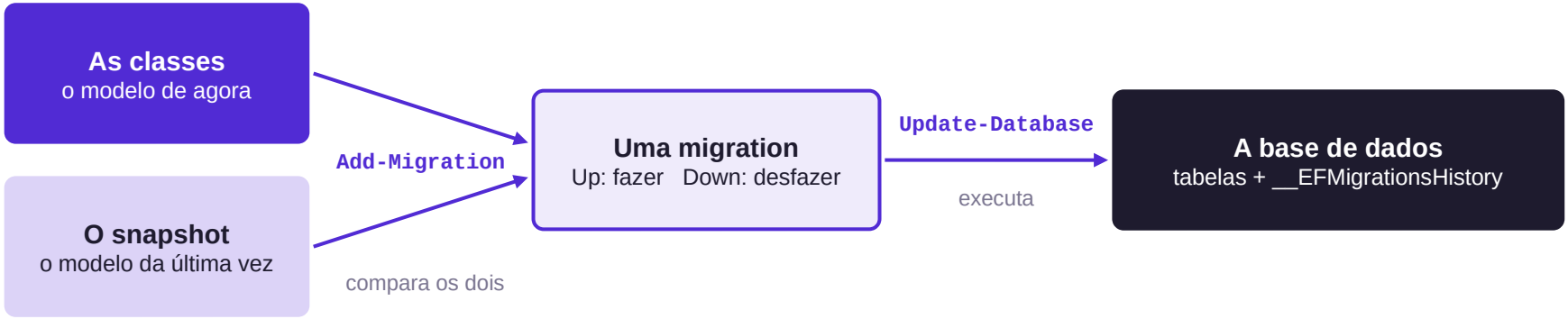
Quando se usa

Quando a base de dados já existe: um sistema antigo, ou uma base de dados gerida por outra equipa.

Nesta cadeira é sempre code first. Não se criam nem alteram tabelas à mão no SQL Server: quem quer mudar a base de dados muda a classe.

ALÍNEA 1.b · MIGRATIONS

Migrations: o que são e como funcionam



A peça	O que guarda
`<data>_BDInicial.cs`	Um passo de alteração: o método Up cria ou altera tabelas, o Down desfaz
`ApplicationDbContextModelSnapshot.cs`	A fotografia do modelo depois da última migration. A próxima compara com ela
`tabela __EFMigrationsHistory`	Na base de dados: a lista das migrations que já foram aplicadas

ALÍNEA 1.b · MIGRATIONS

Migrations: os dois comandos

`Add-Migration BDInicial`

Gera a migration

- Compila o projeto (tem de estar sem erros)
- Compara as classes com o snapshot
- Escreve o ficheiro da migration e atualiza o snapshot
- Não toca na base de dados

`Update-Database`

Aplica as migrations

- Vê na base de dados quais faltam
- Executa-as, pela ordem em que foram criadas
- Cria a base de dados, se ainda não existir
- Regista-as em __EFMigrationsHistory

O que o Add-Migration escreve (excerto do método Up)

```
migrationBuilder.CreateTable(
    name: "Categorias",
    columns: table => new {
        Id = table.Column<int>(nullable: false),
        Nome = table.Column<string>(nullable: true),
        Ordem = table.Column<int>(nullable: false) ...
```

Onde se escrevem

Na Consola do Gerenciador de Pacotes: menu Ferramentas > Gerenciador de Pacotes do NuGet. Remove-Migration desfaz a última, se ainda não foi aplicada.

ALÍNEA 1.b · MIGRATIONS

Migrations: quando as vão usar

Já existe

O template trouxe a primeira migration, em Data/Migrations: cria as tabelas do Identity. Se foi gerada para SQLite, apaga-se na alínea 3.e.

Ficha 3: hoje

Mudamos o modelo: três entidades novas e campos no utilizador. Não se corre nenhum comando.

Ficha 4

Add-Migration BDInicial e Update-Database: nasce a base de dados FrutariaPWeb.

Daí em diante

Sempre que uma classe do modelo mudar: nova migration e novo Update-Database.

A regra

- Mudou uma entidade, o ApplicationUser ou um DbSet: nova migration
- Mudou uma página, um serviço ou o Program.cs: não é preciso
- No trabalho prático é igual

O que não fazer

- Criar ou alterar tabelas à mão no SQL Server
- Editar o ficheiro do snapshot
- Apagar uma migration que já foi aplicada à base de dados

ALÍNEA 1.b · PASTA DATA

Duas classes novas na pasta Data: de onde vieram

ApplicationUser.cs

Diz o que é um utilizador desta aplicação. Cada objeto desta classe é uma conta: uma linha da tabela AspNetUsers.

ApplicationDbContext.cs

Representa a base de dados no código C#. É por ela que se lê e grava tudo: utilizadores, roles e, mais tarde, os nossos dados.

Porque apareceram

Não as escrevemos. O template criou-as porque escolhemos Contas Individuais: o Identity precisa de saber o que é um utilizador e de ter onde o guardar.

São nossas, não da biblioteca

Os serviços do Identity vêm dentro da biblioteca e não se alteram. Estas duas são ficheiros do nosso projeto, criados quase vazios de propósito, para os completarmos.

Já estão ligadas ao resto: o Program.cs e as páginas de conta do template foram escritos em função destas duas classes. Alteramo-las nas alíneas 3.c e 3.d.

ALÍNEA 1.b · PASTA DATA

ApplicationDbContext, linha a linha

GestaoLoja/Data/ApplicationDbContext.cs, como vem do template

```
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace GestaoLoja.Data;

public class ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
    : IdentityDbContext<ApplicationUser>(options)
{
}
```

options

A ligação a usar: que motor de base de dados e que connection string. Quem as entrega é o Program.cs, com AddDbContext e UseSqlServer.

Construtor primário

Os parâmetros logo a seguir ao nome da classe são o construtor (C# 12). Recebe options e passa-as à classe base.

A classe base

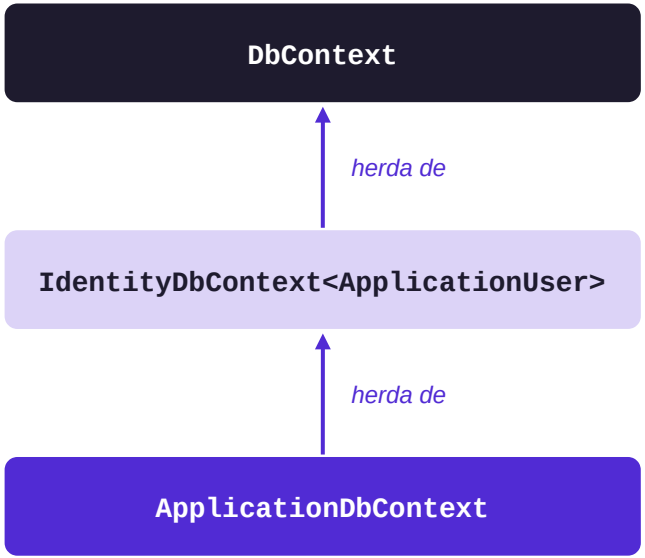
IdentityDbContext já traz as tabelas de utilizadores e roles. Entre < e > vai a nossa classe de utilizador.

Corpo vazio

Ainda sem tabelas nossas, mas já representa as sete tabelas AspNet, que herda. Os DbSet entram na alínea 3.c.

ALÍNEA 1.b · PASTA DATA

IdentityDbContext: o contexto que já traz utilizadores



A base do EF Core: ligação, consultas, SaveChanges.

Acrescenta as tabelas de utilizadores e roles: AspNetUsers, AspNetRoles, AspNetUserRoles e mais quatro.

A nossa classe. Acrescenta Categorias, Produtos e ModosEntrega.

Sem Identity a classe base seria DbContext e as tabelas de utilizadores teriam de ser escritas por nós.

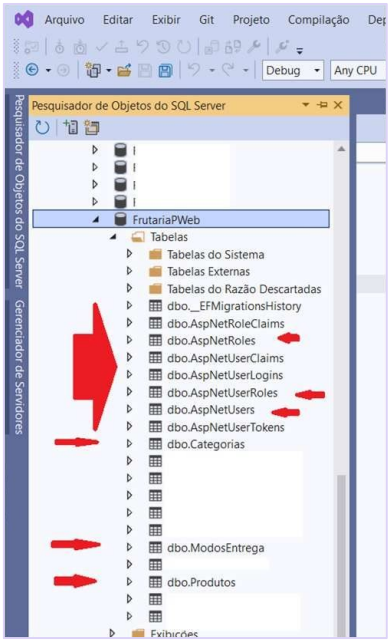


Figura 5 da ficha 4: as tabelas que vão nascer.

ALÍNEA 1.b · PASTA DATA

ApplicationUser, linha a linha

GestaoLoja/Data/ApplicationUser.cs, como vem do template

```
using Microsoft.AspNetCore.Identity;

namespace GestaoLoja.Data;

// Add profile data for application users by
// adding properties to the ApplicationUser class
public class ApplicationUser : IdentityUser
{
}
```

Porque não usar IdentityUser diretamente?

É uma classe da biblioteca: não lhe podemos acrescentar campos. Com uma classe nossa pelo meio, somar Nome ou NIF é só escrever a propriedade.

1 **`using ...Identity;`**

Onde está a classe IdentityUser, que vem com a biblioteca.

2 **`namespace GestaoLoja.Data;`**

O ficheiro está na pasta Data. Com ponto e vírgula, vale para o ficheiro inteiro.

3 **`: IdentityUser`**

Herança: a classe já nasce com Id, UserName, Email, PasswordHash e o resto.

4 **`{ }`**

Vazia de propósito: é aqui que acrescentamos os nossos campos (alínea 3.d).

O resto da aplicação já usa ApplicationUser, por isso nada mais muda.

ETAPA 2 · RELAÇÕES

O problema: como se liga um curso à sua categoria

Cursos

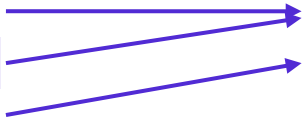
Id	Nome	CategoriaId
1	C# Básico	1
2	Blazor	1
3	SQL Server	2

chave estrangeira (FK)

Categorias

Id	Nome
1	Programação
2	Bases de Dados

chave primária (PK)



Na base de dados, a ligação é um número

Cada linha de Cursos guarda, na coluna CategoriaId, o Id da categoria a que pertence. É essa coluna, a chave estrangeira, que faz a relação.

Um para muitos (1-N)

Uma categoria tem muitos cursos; cada curso tem uma categoria. A chave estrangeira fica sempre do lado "muitos". Há outros tipos de relação (1-1 e N-N), que ficam para mais tarde.

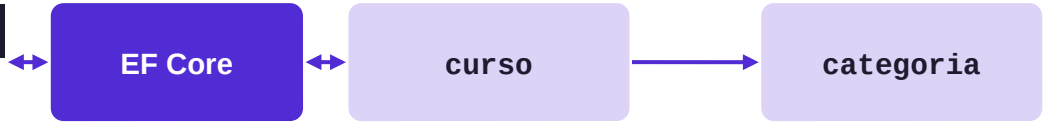
ETAPA 2 · RELAÇÕES

Dois mundos: números nas tabelas, objetos no C#

Na base de dados

Id	Nome	CategoriaId
2	Blazor	1

A ligação é o valor 1.



No C#

A ligação é uma referência a um objeto.

Com propriedades de navegação, vai-se de um objeto ao outro sem escrever o JOIN

```
curso.categoria.Nome          // "Programação": do curso para a sua categoria
categoria.Cursos.Count        // 2: da categoria para os seus cursos
```

Propriedade de navegação

Uma propriedade cujo tipo é outra entidade, ou uma coleção de outra entidade.

Quem a preenche

O EF Core, ao ler da base de dados, a partir do valor da chave estrangeira.

ETAPA 2 · RELAÇÕES

O vocabulário das relações

Entidade principal (lado 1)

```
public class Categoria {
    public int Id { get; set; }
    public string Nome { get; set; }
    public ICollection<Curso> Cursos { get; set; }
}
```

Entidade dependente (lado N)

```
public class Curso {
    public int Id { get; set; }
    public string Nome { get; set; }
    public int CategoriaId { get; set; }
    public Categoria categoria { get; set; }
}
```

Termo	O que é	No exemplo
Entidade principal	A que tem a chave que as outras referem	`Categoria`
Entidade dependente	A que guarda a chave estrangeira	`Curso`
Chave principal	Identifica cada linha da entidade principal	`Categoria.Id`
Chave estrangeira	Guarda, no dependente, o valor da chave principal	`Curso.CategoriaId`
Navegação de referência	Aponta para um único objeto relacionado	`curso.categoria`
Navegação de coleção	Aponta para muitos objetos relacionados	`categoria.Cursos`

ETAPA 2 · CONVENÇÕES

Convenções: o EF Core deduz o que não escrevemos

O que escrevemos	O que o EF Core deduz
`public int Id`	É a chave primária, com numeração automática
`DbSet<Curso> Cursos`	Existe uma tabela chamada Cursos
`string Nome` / `string? Nome`	Coluna de texto obrigatória / que aceita NULL
`public Categoria categoria`	O tipo é outra entidade: há uma relação
`public int CategoriaId`	Pelo nome, é a chave estrangeira dessa relação

Schema

Por omissão, todos os objetos são criados no schema dbo. É por isso que as tabelas aparecem como dbo.Produtos.

Tabelas com DbSet

Cada DbSet<T> da classe de contexto dá origem a uma tabela.

Tabelas sem DbSet

Uma entidade que não tem DbSet, mas é referida por outra que tem, também ganha tabela.

ETAPA 2 · CONVENÇÃO 1 DE 4

Convenção 1: referência na entidade dependente

O que se escreve

```
public class Curso {
    public int Id { get; set; }
    public string Nome { get; set; }
    public Categoria categoria { get; set; }
}

public class Categoria {
    public int Id { get; set; }
    public string Nome { get; set; }
}
```

O que o EF Core deduz

Curso tem uma propriedade do tipo Categoria, que é outra entidade: há uma relação, e Curso é o dependente.

A classe não tem chave estrangeira, por isso o EF cria-a sozinho na tabela: categoriaId (nome da navegação + Id).

A tabela Cursos que resulta

Id	Nome	categoriaId
int	nvarchar	**int, criada pelo EF**

Obrigatória: o tipo Categoria não é anulável, logo todo o curso tem categoria.

O que se consegue fazer no C#

`curso.categoria`	sim	
`categoria.Cursos`	não	não existe
`curso.CategoriaId`	não	não existe

ETAPA 2 · CONVENÇÃO 2 DE 4

Convenção 2: coleção na entidade principal

O que se escreve

```
public class Curso {
    public int Id { get; set; }
    public string Nome { get; set; }
}

public class Categoria {
    public int Id { get; set; }
    public string Nome { get; set; }
    public ICollection<Curso> Cursos { get; set; }
}
```

O que o EF Core deduz

Categoria tem uma coleção de Curso: uma categoria, muitos cursos. Curso é o dependente, mesmo sem ter lá nada escrito.

O EF cria em Cursos a coluna CategoriaId (nome da entidade principal + Id).

A tabela Cursos que resulta

Id	Nome	CategoriaId
int	nvarchar	**int, aceita NULL**

Opcional: nada na classe Curso obriga a ter categoria, por isso a coluna aceita NULL.

O que se consegue fazer no C#

`curso.categoria`	**não**	não existe
`categoria.Cursos`	**sim**	
`curso.CategoriaId`	**não**	não existe

ETAPA 2 · CONVENÇÃO 3 DE 4

Convenção 3: referência e coleção

O que se escreve

```
public class Curso {
    public int Id { get; set; }
    public string Nome { get; set; }
    public Categoria categoria { get; set; }
}

public class Categoria {
    public int Id { get; set; }
    public string Nome { get; set; }
    public ICollection<Curso> Cursos { get; set; }
}
```

O que o EF Core deduz

As duas propriedades apontam uma para a outra: o EF junta-as numa só relação. Cada uma é a navegação inversa da outra.

A chave estrangeira continua a ser criada pelo EF: categoriaId.

A tabela Cursos que resulta

Id	Nome	categoriaId
int	nvarchar	**int, criada pelo EF**

Obrigatória, pela mesma razão da convenção 1.

O que se consegue fazer no C#

`curso.categoria`	sim	
`categoria.Cursos`	sim	
`curso.CategoriaId`	não	não existe

ETAPA 2 · CONVENÇÃO 4 DE 4

Convenção 4: a relação completa

O que se escreve

```
public class Curso {
    public int Id { get; set; }
    public string Nome { get; set; }
    public int CategoriaId { get; set; }
    public Categoria categoria { get; set; }
}

public class Categoria {
    public int Id { get; set; }
    public string Nome { get; set; }
    public ICollection<Curso> Cursos { get; set; }
}
```

O que o EF Core deduz

A chave estrangeira está escrita na classe. O EF reconhece-a pelo nome: Categoria + Id.

Passa a ser uma propriedade como as outras: lê-se e escreve-se. É assim que um formulário grava a categoria escolhida numa lista.

A tabela Cursos que resulta

Id	Nome	CategoriaId
int	nvarchar	**int, propriedade da classe**

Obrigatória porque é int. Com int? passava a opcional.

O que se consegue fazer no C#

`curso.categoria`	sim
`categoria.Cursos`	sim
`curso.CategoriaId`	sim

ETAPA 2 · CONVENÇÕES

As quatro convenções lado a lado

	Convenção 1	Convenção 2	Convenção 3	Convenção 4
Escreve-se em Curso	referência	nada	referência	FK e referência
Escreve-se em Categoria	nada	coleção	coleção	coleção
Coluna criada em Cursos	`categoriaId`	`CategoriaId`	`categoriaId`	`CategoriaId`
Chave estrangeira no C#	não	não	não	sim
Relação obrigatória	sim	não	sim	sim
curso.categoria	sim	não	sim	sim
categoria.Cursos	não	sim	sim	sim

O que não muda

Nas quatro, a tabela Cursos fica com uma coluna que é chave estrangeira para Categorias. O que muda é o que se consegue fazer no C#.

Na loja

Usamos a 4: os formulários precisam da chave estrangeira.

ETAPA 2 · CONVENÇÕES

Como o EF encontra a chave estrangeira

Na classe dependente, o EF procura uma propriedade com um destes nomes:

Regra	No exemplo dos cursos	Na loja
nome da navegação + Id	`categoria + Id`	não é a que usamos
nome da entidade principal + Id	`Categoria + Id`	`CategoriaId, ModoEntregaId`

Se encontrar

É essa a chave estrangeira. O tipo da propriedade decide se a relação é obrigatória (int) ou opcional (int?).

Se não encontrar

Cria uma propriedade sombra: uma coluna que existe na tabela e no modelo do EF, mas não na classe. Foi o que aconteceu nas convenções 1, 2 e 3.

Se o nome fugir à regra

Por exemplo, uma propriedade IdCat. O EF não a reconhece, e é preciso indicá-la com o atributo [ForeignKey]. Fica para mais tarde.

ETAPA 2 · RELAÇÕES

Obrigatória ou opcional: int ou int?

```
public int CategoriaId { get; set; }
```

Id	Nome	CategoriaId
1	C# Básico	1
2	Blazor	1
3	SQL Server	2

```
public int? CategoriaId { get; set; }
```

Id	Nome	CategoriaId
1	C# Básico	1
2	Blazor	NULL
3	SQL Server	2

Relação obrigatória

- A coluna é NOT NULL: não há curso sem categoria
- Apagar uma categoria apaga os seus cursos (em cascata)
- Na loja: a categoria de um produto

Relação opcional

- A coluna aceita NULL: pode haver cursos sem categoria
- Apagar uma categoria não apaga os seus cursos
- Na loja: o modo de entrega de um produto

ETAPA 3 · MAPA

Mapa da etapa 3, alínea a alínea

Alínea	Ação	O quê	Onde (a partir da pasta do projeto)
3.a	**Apagar**	Counter, Weather e Error (.razor) e as duas entradas do menu	`Components/Pages/ e Layout/`
3.b	**Criar**	Categoria.cs, ModoEntrega.cs, Produto.cs	`Entities/`
3.c	**Alterar**	ApplicationDbContext.cs: três DbSet	`Data/`
3.d	**Alterar**	ApplicationUser.cs: campos do utilizador	`Data/`
3.e	**Alterar**	Confirmar o motor; appsettings.json: Database=FrutariaPWeb	`(raiz do projeto)`
3.g	**Criar**	Inicializacao.cs: roles e administrador	`Data/`
3.h	**Criar**	AppConfig.cs: textos fixos da loja	`(raiz do projeto)`
3.i	**Alterar**	Program.cs: AddRoles e AddHttpContextAccessor	`(raiz do projeto)`
3.k	**Alterar**	Program.cs: chamar os dados iniciais	`(raiz do projeto)`
3.l	**Alterar**	_Imports.razor: dois @using	`Components/`
3.n	**Criar**	pasta img com as imagens da interface	`wwwroot/img/`

Alíneas 3.f, 3.j e 3.m: compilar. Alínea 3.o: guardar no disco as imagens de exemplo, para a ficha 5.

ALÍNEA 3.a · AUTHORIZEVIEW

AuthorizeView: o menu muda com quem está a ver

Components/Layout/NavMenu.razor (simplificado)

```
<AuthorizeView>
    <Authorized>
        <!-- com sessão iniciada -->
        @context.User.Identity?.Name
        Logout
    </Authorized>
    <NotAuthorized>
        <!-- sem sessão iniciada -->
        Register   Login
    </NotAuthorized>
</AuthorizeView>
```

Sem sessão iniciada

Aparece o que está em NotAuthorized: as ligações Register e Login.

Com sessão iniciada

Aparece o que está em Authorized: o nome do utilizador, que vem em context, e o Logout.

Como sabe quem está a ver

Depois do login, um cookie identifica o utilizador em cada pedido. O Program.cs entrega esse estado a todos os componentes.

Esconder não é proteger

AuthorizeView só decide o que se mostra. Quem impede a entrada numa página é @attribute [Authorize], como na Auth.razor.

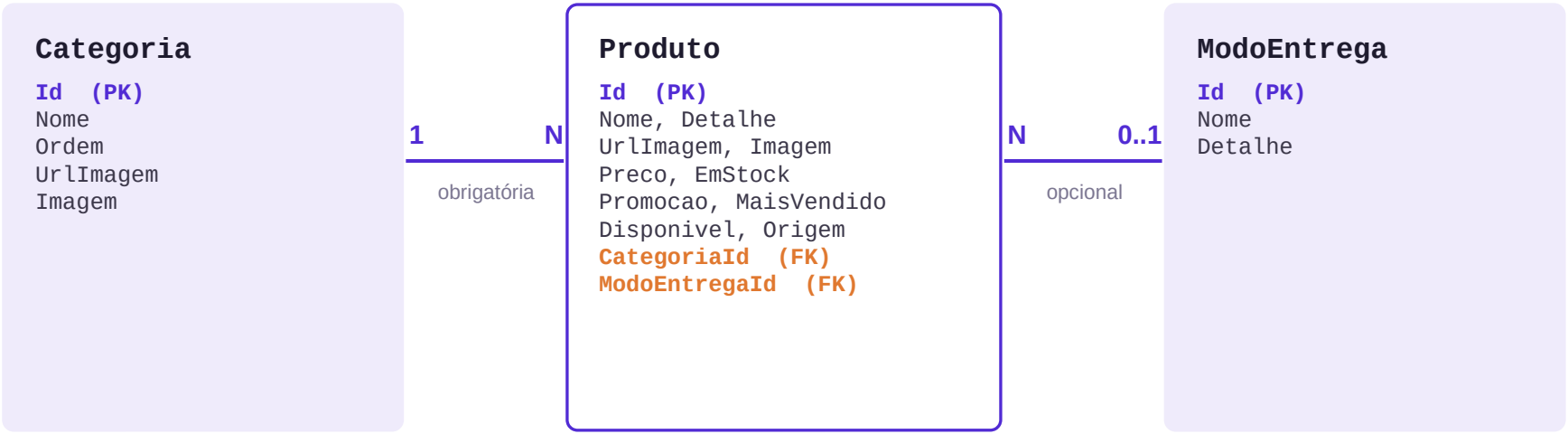
Também por role

Pode restringir-se a quem tem um certo role. O menu da loja, na ficha 5, volta a usar este componente.

```
<AuthorizeView Roles="Admin">
```

ALÍNEA 3.b · ENTIDADES

O modelo de dados da loja



Categoria e Produto

Uma categoria tem muitos produtos. Todo o produto tem categoria: a chave estrangeira não aceita NULL.

ModoEntrega e Produto

Um modo de entrega (quilo, litro, unidade, pack) serve muitos produtos. Um produto pode não ter modo de entrega.

ALÍNEA 3.b · ENTIDADES

As duas relações, tal como as escrevemos

Entities/Produto.cs e Entities/ModoEntrega.cs (excertos)

```
public class Produto
{
    // ... Id, Nome, Preco, ...

    public int CategoriaId { get; set; }
    public Categoria categoria { get; set; }

    public int? ModoEntregaId { get; set; }
    public ModoEntrega modoentrega { get; set; }
}

public class ModoEntrega
{
    // ... Id, Nome, Detalhe
    public ICollection<Produto>? produtos { get; set; }
}
```

Produto e Categoria

- Chave estrangeira int e referência, em Produto
- Nada em Categoria: não precisamos de ir da categoria aos produtos
- Obrigatória

Produto e ModoEntrega

- Chave estrangeira int? e referência, em Produto
- Coleção em ModoEntrega
- É a convenção 4 completa, mas opcional

ALÍNEAS 3.b E 3.d · ANOTAÇÕES

Anotações: validar o formulário e moldar a tabela

Anotação	No formulário	Na tabela
`[Required]`	campo obrigatório	coluna NOT NULL
`[StringLength(100)]`	no máximo 100 caracteres	nvarchar(100)
`[RegularExpression(...)]`	formato obrigatório	nada
`[Column(TypeName = "decimal(10,2)")]`	nada	tipo exato da coluna
`[NotMapped]`	nada	a propriedade não tem coluna
`[JsonIgnore]`	nada	nada: só afeta a conversão em JSON

Três propriedades para uma imagem (em Categoria e em Produto)

`UrlImagem`
`string`
O nome do ficheiro.

`Imagem`
`byte[]`
A imagem em si, guardada na base de dados.

`ImageFile`
`IFormFile, [NotMapped]`
O ficheiro em trânsito, enquanto é carregado no formulário.

ALÍNEA 3.c · CONTEXTO

DbContext e DbSet: a base de dados vista do C#

GestaoLoja/Data/ApplicationDbContext.cs

```
using GestaoLoja.Entities;

public class ApplicationDbContext(...)
    : IdentityDbContext<ApplicationUser>(options)
{
    public DbSet<Categoria>   Categorias   { get; set; }
    public DbSet<Produto>     Produtos     { get; set; }
    public DbSet<ModoEntrega> ModosEntrega { get; set; }
}
```

DbContext

A sessão com a base de dados: abre a ligação, traduz as consultas e guarda as alterações.

DbSet<T>

Uma tabela. T é a classe que descreve cada linha.

O nome

O nome da propriedade passa a ser o nome da tabela.

É um serviço: o contexto é registado no Program.cs (AddDbContext) e chega às páginas por injeção de dependências, como os serviços da ficha 2. É também pelos DbSet que se leem e gravam dados: context.Produtos.ToList().

ALÍNEA 3.d · UTILIZADOR

ApplicationUser: acrescentar campos ao utilizador

`IdentityUser`

Já existe. Id, UserName, Email, PasswordHash, PhoneNumber, EmailConfirmed, ...

herda de

`ApplicationUser`

É nosso. Nome, Apelido, Rua, Localidade, Cidade, CodigoPostal, Pais, Telemovel, NIF, Fotografia, Estado, DataRegisto

uma só tabela: AspNetUsers

Validação declarada na classe

`[StringLength(100)]` tamanho máximo

`[RegularExpression]` formato do código postal e do telemóvel

`[validarNIF]` atributo escrito por nós: herda de ValidationAttribute e confere o dígito de controlo

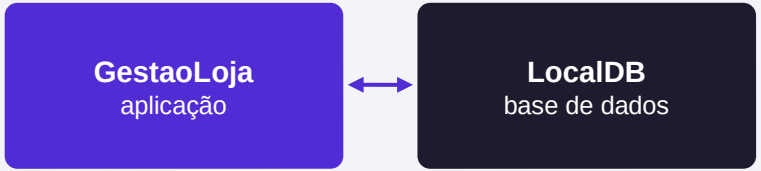
Porquê herdar em vez de criar outra tabela

O login, o hash da palavra-passe e os roles continuam a funcionar sem escrevermos código. Os nossos campos ficam na mesma linha do utilizador.

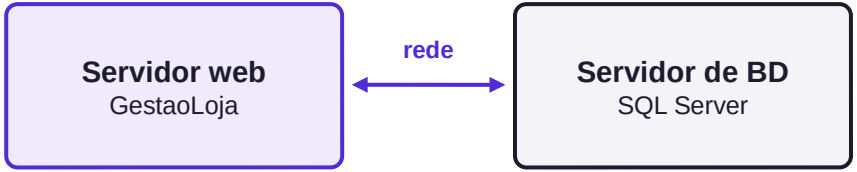
ALÍNEA 3.e · BASE DE DADOS

Na mesma máquina hoje, noutra máquina amanhã

NAS AULAS: UM COMPUTADOR



NUMA EMPRESA: MÁQUINAS SEPARADAS



A morada: uma instância local

```
Server=(localdb)\mssqllocaldb;
Database=FrutariaPWeb;Trusted_Connection=True
```

A morada: outra máquina, e credenciais para entrar

```
Server=bd.loja.pt,1433;Database=FrutariaPWeb;
User Id=loja_app;Password=*******
```

O código não muda. Entidades, contexto e páginas ficam iguais; só muda a connection string. É por isso que ela vive num ficheiro de configuração e não está escrita no C#.

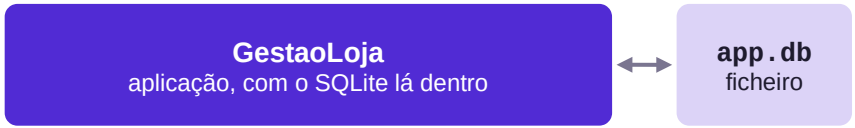
ALÍNEA 3.e · BASE DE DADOS

SQL Server ou SQLite: confirmar o motor

SQL SERVER LOCALDB: UM SERVIDOR



SQLITE: UM FICHEIRO



O que muda	SQLite	SQL Server
Pacote	`...Core.Sqlite`	`...Core.SqlServer`
Program.cs	`UseSqlite`	`UseSqlServer`
Connection string	`DataSource=...`	`Server=(localdb)...`
Migrations	para SQLite	para SQL Server

Como saber qual veio

- Abrir o appsettings.json e ler DefaultConnection
- Começa por Server=(localdb): SQL Server. Só muda o nome da base de dados
- Começa por DataSource=: SQLite. Passos 2 a 4 da alínea 3.e do guia

O que não depende do motor

- As entidades, o ApplicationDbContext e as páginas ficam iguais
- É o pacote que traduz o LINQ para o SQL de cada motor
- Nesta cadeira usa-se sempre SQL Server LocalDB

ALÍNEA 3.e · CONNECTION STRING

appsettings.json e a connection string

GestaoLoja/appsettings.json

```
"DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=FrutariaPWeb;
                     Trusted_Connection=True;MultipleActiveResultSets=true"
```

Server

O servidor: a instância mssqllocaldb do SQL Server LocalDB, na própria máquina.

Database

O nome da base de dados: FrutariaPWeb, o que aparece nas figuras da ficha 4.

Trusted_Connection

Entra-se com o utilizador do Windows. Não há palavra-passe no ficheiro.

MultipleActiveResultSets

Permite várias leituras abertas na mesma ligação.

Como chega ao contexto

`appsettings.json` → `GetConnectionString("DefaultConnection")` → `AddDbContext(UseSqlServer)` → **`ApplicationDbContext`**

ALÍNEA 3.e · BOA PRÁTICA

Palavra-passe no appsettings.json: o problema

appsettings.json: o que não se deve fazer

```
"ConnectionStrings": {
  "DefaultConnection":
    "Server=bd.loja.pt;Database=FrutariaPWeb;
     User Id=loja_app;Password=Fruta2026!"
}
```

Porquê

O ficheiro faz parte do projeto e vai para o Git. Quem tiver acesso ao repositório fica com a palavra-passe, e ela continua no histórico mesmo depois de apagada.

E a nossa connection string?

Não tem palavra-passe: Trusted_Connection usa a conta do Windows. Pode ficar no appsettings.json sem problema.

O que conta como segredo

Palavras-passe de bases de dados, chaves de API, credenciais de email, chaves de assinatura de tokens.

A regra: um segredo nunca entra no repositório. Não usamos isto nesta cadeira, mas é assim que se trabalha fora dela.

ALÍNEA 3.e · BOA PRÁTICA

A forma certa: os segredos ficam fora do projeto

De onde o .NET lê a configuração (a última origem ganha)

appsettings.json
valores normais, vão para o Git

▼

appsettings.Development.json
o que só muda em desenvolvimento

▼

User Secrets
segredos em desenvolvimento, fora do projeto

▼

Variáveis de ambiente
segredos em produção, definidos no servidor

User Secrets, em três passos

1. Clique direito no projeto > Gerenciar Segredos do Usuário. Abre um ficheiro secrets.json.

2. Escrever lá a connection string com a palavra-passe, com a mesma estrutura do appsettings.json.

3. Mais nada. O código continua a pedir DefaultConnection e recebe o valor do secrets.json.

O ficheiro fica na pasta do utilizador do Windows, não na do projeto. Não é cifrado: serve só para desenvolvimento.

O mesmo, pela linha de comandos, na pasta do projeto

```
dotnet user-secrets set "ConnectionStrings:DefaultConnection" "Server=...;Password=..."
```

ALÍNEAS 3.g A 3.k · O ARRANQUE

Program.cs: registar, construir, executar

1. Registar serviços → **2. Construir** → **3. Atender pedidos**

`builder.Services...`

```
AddRazorComponents
AddAuthentication
AddDbContext
AddIdentityCore
```

+ AddRoles (3.i)
+ AddHttpContextAccessor (3.i)

`var app = builder.Build();`

Os serviços passam a existir e podem ser pedidos.

+ dados iniciais (3.k)

`app.Use...  app.Run();`

```
UseHttpsRedirection
UseStaticFiles
MapRazorComponents
Run: fica à espera
```

A ordem conta

AddRoles tem de ficar antes de AddEntityFrameworkStores. É este último que liga utilizadores e roles às tabelas, e só liga os roles se já souber que existem.

Sem AddRoles

Compila, mas falha ao arrancar: o RoleManager não está registado.

ALÍNEAS 3.g E 3.i · IDENTITY

Os três Managers: de onde vêm e onde os usamos

GestaoLoja/Program.cs: é aqui que ficam disponíveis

```
builder.Services
  .AddIdentityCore<ApplicationUser>(...)    // UserManager
  .AddSignInManager()                       // SignInManager
  .AddRoles<IdentityRole>()                 // RoleManager (3.i)
  .AddDefaultTokenProviders()
  .AddEntityFrameworkStores<ApplicationDbContext>();
```

São do Identity, não nossos

As três classes vêm com a biblioteca. Não as escrevemos nem as alteramos: usamos os métodos que já trazem.

Quem os põe a funcionar

O Program.cs. Cada linha da cadeia regista um serviço no contentor de injeção de dependências, como na ficha 2. O template regista dois; o RoleManager acrescentamos nós.

Como se obtêm

Pedem-se ao contentor, não se criam com new. Numa página:

```
@inject UserManager
  <ApplicationUser> UserManager
```

Onde os usamos

- UserManager e RoleManager: na classe Inicializacao (alínea 3.g)
- SignInManager e UserManager: nas páginas Login e Register do template

ALÍNEA 3.g · IDENTITY

Dos serviços às tabelas

Serviço	Métodos que vamos ver	Tabelas em que mexe
`UserManager`	`CreateAsync, FindByEmailAsync, AddToRoleAsync`	`AspNetUsers, AspNetUserRoles`
`SignInManager`	`PasswordSignInAsync, SignOutAsync`	lê AspNetUsers e cria o cookie de sessão
`RoleManager`	`RoleExistsAsync, CreateAsync`	`AspNetRoles`

O caminho de uma chamada

Página ou Inicializacao
pede o serviço

→

Manager
regras do Identity

→

Store
acesso com EF Core

→

ApplicationDbContext
o nosso contexto

→

SQL Server
as tabelas

Os Managers não escrevem SQL

Usam o nosso ApplicationDbContext, por intermédio dos stores que AddEntityFrameworkStores regista.

A palavra-passe

CreateAsync recebe-a em texto, calcula o hash e guarda o hash na coluna PasswordHash.

ALÍNEAS 3.g E 3.k · DADOS INICIAIS

Dados iniciais: os roles e o administrador

1. **A aplicação arranca**
   Depois do Build, antes de atender pedidos.

2. **Pede os dois serviços**
   Cria um scope e pede o UserManager e o RoleManager.

3. **Cria os roles**
   Admin, Gestor e Cliente, se ainda não existirem.

4. **Cria o administrador**
   Se ainda não existir, e dá-lhe o role Admin.

Credenciais de arranque (para a ficha 5)

`admin@localhost.com` `Is3C..00`

Porquê no arranque

A base de dados nasce vazia. Sem administrador, ninguém conseguia entrar para criar os outros utilizadores.

Porquê "se não existir"

O código corre sempre que a aplicação arranca. Sem a verificação, tentava criar os mesmos dados outra vez.

Porquê um scope

UserManager e RoleManager são Scoped: vivem durante um pedido. No arranque não há pedido, por isso cria-se o scope à mão.

Quem chama a Inicializacao, e quando

GestaoLoja/Program.cs, resumido

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<...>(...);        // 1
builder.Services.AddIdentityCore<...>(...)
    .AddRoles<IdentityRole>() ...;
var app = builder.Build();                      // 2
using (var scope = app.Services.CreateScope()) // 3
{
    var userManager = ...GetRequiredService<...>();
    var roleManager = ...GetRequiredService<...>();
    await Inicializacao.CriaDadosIniciais(      // 4
        userManager, roleManager);
}
app.UseHttpsRedirection(); ...                  // 5
app.Run();                                      // 6
```

Criada na 3.g, chamada na 3.k

A `Inicializacao` é uma classe static na pasta `Data`. Sozinha não corre: sem a linha 4, compilava e nunca era executada.

1. **`builder.Services.Add...`**
   Regista os serviços. Ainda nada corre.
2. **`builder.Build()`**
   Os serviços passam a existir.
3. **`CreateScope()`**
   Pede o UserManager e o RoleManager.
4. **`Inicializacao.CriaDadosIniciais`**
   Cria roles e admin. O await espera pelo fim.
5. **`app.Use...` `app.Map...`**
   Monta o caminho de cada pedido HTTP.
6. **`app.Run()`**
   Atende pedidos até a aplicação parar.

ALÍNEA 3.k · ATENÇÃO

Porque é que a aplicação deixa de arrancar

Até à alínea 3.j

A aplicação arranca. Ao arrancar, nada consulta a base de dados.

→

Alínea 3.k

O arranque passa a chamar os dados iniciais, que consultam a base de dados.

→

Resto da ficha 3

A base de dados FrutariaPWeb não existe. Com F5, o arranque termina numa exceção.

→

Ficha 4

Update-Database cria a base de dados. A aplicação volta a arrancar e cria o administrador.

A exceção que aparece no Visual Studio

```
SqlException: Cannot open database "FrutariaPWeb" requested by the login. The login failed.
```

O que fazer

Nada. Compilar com Ctrl+Shift+B e continuar: a ficha só pede para compilar.

O que não fazer

Apagar o bloco dos dados iniciais para "resolver". O código está certo.

ALÍNEAS 3.h, 3.l E 3.n

Três peças pequenas, a pensar na ficha 5

AppConfig.cs

Uma classe estática com os textos fixos da loja: título, morada, contactos. Mudam-se num só sítio.

```
@AppConfig.tituloHomePage
```

_Imports.razor

Os @using comuns a todos os componentes da pasta. Acrescentamos os namespaces das entidades e do contexto.

```
@using GestaoLoja.Data
@using GestaoLoja.Entities
```

wwwroot/img

Ficheiros estáticos: são entregues ao browser tal como estão. O endereço não inclui wwwroot.

```
wwwroot/img/novo.png
→  /img/novo.png
```

Nenhuma das três tem efeito visível hoje. Ficam prontas para as páginas da ficha 5: o título no menu, os @using das páginas de gestão e os ícones dos botões.

ETAPA 3 · RESULTADO

O projeto no fim da ficha 3

GestaoLoja/ (pasta do projeto)

```
├─ Components/
│   ├─ Layout/NavMenu.razor
│   ├─ Pages/          Counter, Weather, Error: apagados
│   └─ _Imports.razor
├─ Data/
│   ├─ Migrations/     apagada, se veio com SQLite
│   ├─ ApplicationDbContext.cs
│   ├─ ApplicationUser.cs
│   └─ Inicializacao.cs
├─ Entities/
│   ├─ Categoria.cs
│   ├─ ModoEntrega.cs
│   └─ Produto.cs
├─ wwwroot/img/       12 imagens
├─ AppConfig.cs
├─ appsettings.json
└─ Program.cs
```

novo | alterado | apagado

Antes de sair

- Compila com zero erros
- Três classes em Entities, com namespace GestaoLoja.Entities
- Três DbSet no contexto
- UseSqlServer no Program.cs; (localdb) e FrutariaPWeb no appsettings.json
- AddRoles antes de AddEntityFrameworkStores
- Bloco dos dados iniciais logo depois do Build
- Doze imagens em wwwroot/img

QUANDO ALGO FALHA

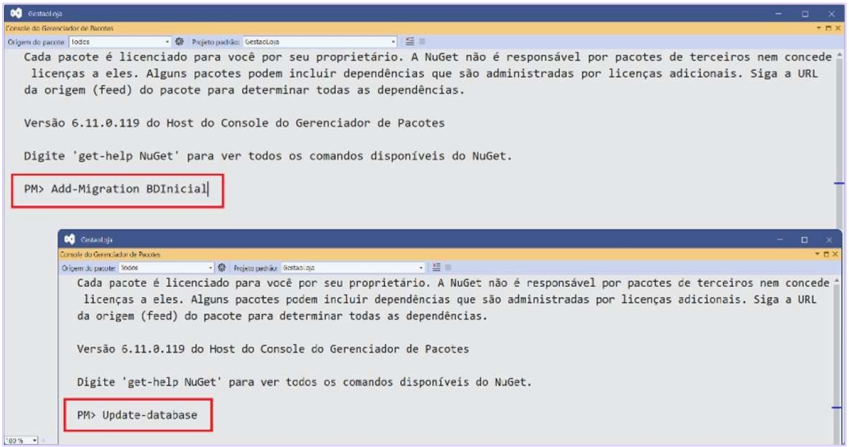
Erros frequentes

Sintoma	Causa e solução
CS0246: Produto não encontrado, em ModoEntrega.cs	Produto.cs ainda não foi criado. Acabar a alínea 3.b.
CS0246: Categoria não encontrado, no contexto	Falta using GestaoLoja.Entities, ou as classes têm outro namespace.
CS0117: ApplicationUser não tem Nome	A alínea 3.d não está feita.
CS1061: UseSqlServer não existe, em Program.cs	O projeto veio com SQLite e falta o pacote do SQL Server. Alínea 3.e, passo 2.
Erros de namespace em todos os ficheiros	O projeto não se chama GestaoLoja. Criar de novo.
Avisos CS8618 em categoria e modoentrega	Esperados. São avisos, não erros.
Ao executar: RoleManager não registado	Falta AddRoles, ou está depois de AddEntityFrameworkStores.
Ao executar: Cannot open database	Esperado depois da alínea 3.k. Resolve-se na ficha 4.
Counter e Weather ainda no menu, com Not Found	Faltou tirar as duas entradas do NavMenu.razor (alínea 3.a).

A SEGUIR

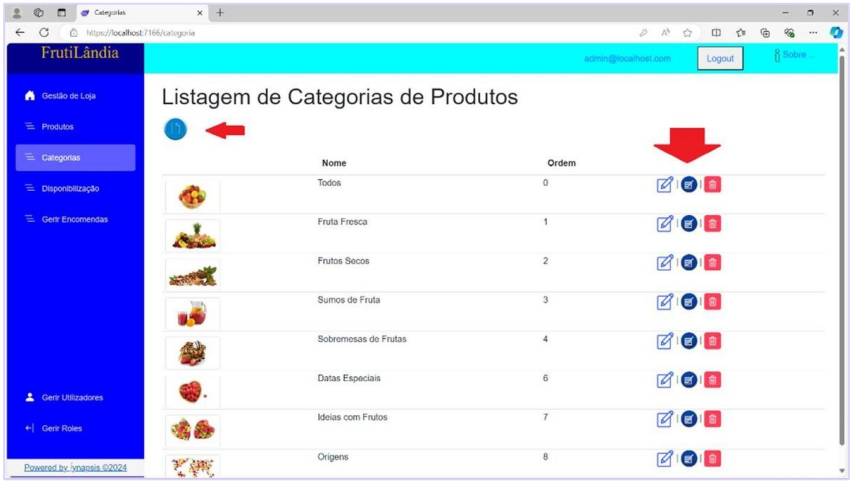
O que vem a seguir: fichas 4 e 5

Ficha 4: a base de dados



`Add-Migration BDInicial` gera o script das tabelas.
`Update-Database` executa-o: nasce a FrutariaPWeb.

Ficha 5: as páginas



O Visual Studio gera as páginas de listar, criar, editar e apagar a partir das classes de hoje.

PARA LEVAR DA AULA

Das classes à base de dados

Identity

Registo, login e roles prontos a usar. Chega com a opção Contas Individuais.

IdentityDbContext

O contexto herda as tabelas de utilizadores. Nós acrescentamos as nossas com DbSet.

ApplicationUser

Herda de IdentityUser. Os campos novos são colunas da mesma tabela.

Relações 1-N

A chave estrangeira fica no lado "muitos". As navegações levam de um objeto ao outro.

Convenções

O EF deduz a relação pela forma das classes. A convenção 4 deixa a chave estrangeira à mão.

Arranque

Registar, construir, executar. Os dados iniciais correm depois do Build e precisam da base de dados.